

RELATÓRIO | APLICAÇÃO DE PADRÕES EM PROJETO PRÁTICO EM MONGODB

27 de maio de 2026

Bruno Rafael Ramos Prata (50056472)

1. ENQUADRAMENTO

O presente relatório descreve a modelação, implementação e análise de uma solução MongoDB para um cenário simples de e-commerce, tendo como objetivo armazenar dados de produtos, utilizadores e compras e, a partir desses dados, gerar recomendações personalizadas. O enunciado do *case study* exige a aplicação de padrões de modelação de dados, a criação de índices, a utilização de agregações e uma reflexão sobre escalabilidade e sharding.

A solução foi construída sobre três coleções principais – *Products*, *Users* e *Purchases* – em conformidade com a estrutura de dados fornecida para o exercício.

Este trabalho procurou responder à questão central do *case study*: como modelar os dados de uma plataforma de e-commerce em MongoDB e implementar um sistema simples de recomendação personalizado, usando padrões de modelação, índices e agregações.

De forma mais concreta, os objetivos foram:

- a. Modelar os dados de produtos, utilizadores e histórico de compras em MongoDB.
- b. Justificar o uso de referências, documentos embutidos e desnormalização.
- c. Implementar recomendações por conteúdo, popularidade e segmento com recurso a agregações.
- d. Criar índices coerentes com as consultas realizadas e discutir o seu impacto no desempenho.
- e. Refletir sobre a escalabilidade da solução e sobre uma possível estratégia de sharding.

2. MODELAÇÃO DE DADOS

2.1. Estrutura geral

A solução foi organizada em três coleções principais: *Products*, *Users* e *Purchases*. Esta separação acompanha diretamente as entidades centrais do domínio de e-commerce e está alinhada com os ficheiros de entrada fornecidos no enunciado.

A coleção *Products* armazena o catálogo de produtos. Cada documento inclui informação descritiva e comercial, como identificador, nome, categoria, preço, marca, estoque, fornecedor, tags, avaliação média e número de avaliações.

A coleção *Users* representa os perfis dos utilizadores. Cada documento inclui identificador, nome, email, preferências, segmento, dados de conta, endereços e um histórico resumido de compras.

A coleção *Purchases* armazena o histórico transacional. Cada compra contém um identificador próprio, referência ao utilizador, referência ao produto, categoria do produto, segmento do utilizador, data da compra, quantidade, preço, método de pagamento, canal, dispositivo e dados de envio.

2.2. Padrões de modelação adotados

A modelação adotada recorreu a referências para relacionar compras com utilizadores e produtos. Em *Purchases*, os campos *user_id* e *produto_id* ligam cada compra ao respetivo utilizador e ao respetivo produto, evitando duplicar documentos completos em cada transação.

Foram igualmente utilizados documentos embutidos em situações em que a informação pertence logicamente à entidade principal e é normalmente consultada em conjunto com ela. Nos dados fornecidos, isso verifica-se em estruturas como *fornecedor* e *dimensoes_mm* em *Products*, *enderecos* e *historico_compras* em *Users*, e *dispositivo* e *shipping* em *Purchases*.

A solução também permite discutir desnormalização e Extended Reference Pattern. Em *Purchases*, os campos *produto_categoria* e *user_segmento* guardam uma fotografia do contexto da compra no momento da transação, permitindo análises futuras sem necessidade de recalculer o estado histórico a partir das coleções principais.

Adicionalmente, o campo *historico_compras* presente em *Users* constitui um exemplo claro de Computed Pattern, já que armazena valores pré-calculados como *total_pedidos*, *gasto_total*, *ultimo_pedido* e *produtos_frequentes*, reduzindo o custo de consultas frequentes sobre comportamento do utilizador.

Por fim, a solução é compatível com uma extensão futura baseada em Subset Pattern, através da manutenção, no documento do utilizador, de um subconjunto de informação frequentemente usada, como últimos produtos comprados ou produtos mais frequentes, em vez de embutir o histórico completo de compras.

2.3. Decisões de modelação e validação do esquema

Durante a implementação, foi necessário validar o esquema real importado em MongoDB, uma vez que pequenas diferenças no nome dos campos tiveram impacto direto nas agregações. A inspeção dos documentos mostrou, por exemplo, que *Products* utiliza *_id* como identificador do produto e *quantidade_estoque* como campo de estoque.

A inspeção do histórico de compras mostrou que *Purchases* utiliza os campos *user_id* e *produto_id*, bem como *produto_categoria* e *user_segmento*, pelo que a pipeline final teve de ser ajustada de acordo com esses nomes reais.

Esta validação foi crítica para o sucesso da solução, pois o MongoDB exige correspondência exata entre nomes de campos em estágios como *\$lookup*, *\$addFields* e *\$match*. Uma diferença mínima, como *quantidade_estoque* versus *quantidadeestoque*, é suficiente para fazer uma consulta devolver resultados vazios.

3. IMPLEMENTAÇÃO DA SOLUÇÃO

3.1. Criação da base de dados e importação

A base de dados foi organizada com três coleções principais, conforme sugerido no enunciado: *Products*, *Users* e *Purchases*. Esta estrutura reflete diretamente os ficheiros JSON fornecidos para o exercício.

Após a importação, foi feita a validação dos dados através da contagem de documentos e da inspeção de documentos individuais com *findOne()*. Esta etapa revelou a estrutura efetiva dos dados e permitiu corrigir a pipeline de recomendação de forma consistente com o esquema real importado.

3.2. Recomendação personalizada baseada em conteúdo

A primeira recomendação implementada foi uma recomendação baseada em conteúdo, centrada no utilizador e nas suas preferências. A lógica adotada segue a sugestão do enunciado: escolher um utilizador, identificar as categorias preferidas, obter o histórico de compras, excluir os produtos já adquiridos e recomendar produtos com estoque dentro das categorias preferidas.

A pipeline foi construída sobre a coleção *Users*, começando com um *\$match* para selecionar o utilizador, seguido de um *\$lookup* para recolher as compras associadas através de *user_id*. Depois, com *\$addFields*, foi criado o array *produtosComprados*, a partir do campo *produto_id* existente em *Purchases*.

Num segundo *\$lookup*, a coleção *Products* foi filtrada com base em três critérios: a categoria do produto tinha de pertencer ao array *preferencias*, o produto não podia constar de *produtosComprados*, e *quantidade_estoque* tinha de ser superior a zero. Para ordenar os resultados, foram usados *avaliacao_media* e *numero_avaliacoes*, priorizando produtos bem avaliados e com maior robustez estatística.

O teste com o utilizador *u1* mostrou que a pipeline devolveu as recomendações *p2* e *p8*, ambas da categoria *Roupas*, coerentes com as preferências de *u1* (*Roupas* e *Calçado*) e excluindo corretamente os produtos já comprados (*p1*, *p3* e *p9*).

The screenshot shows the MongoDB Compass interface. On the left, the 'CaseStudy | E-commerceDB' database is selected, and the 'Users' collection is highlighted. The main panel displays an aggregation pipeline with the following stages: *\$match*, *\$lookup*, *\$addFields*, *\$lookup*, and *\$project*. The results are shown in a JSON format:

```
{
  "_id": "u1",
  "nome": "Ana Sousa",
  "preferencias": [
    "Roupas",
    "Calçado"
  ],
  "produtosComprados": [
    "p1",
    "p3",
    "p9"
  ],
  "recomendacoes": [
    {
      "_id": "p2",
      "nome": "Calças Ganga",
      "categoria": "Roupas",
      "marca": "DenimCraft",
      "quantidade_estoque": 120,
      "avaliacao_media": 4.7,
      "numero_avaliacoes": 86
    },
    {
      "_id": "p8",
      "nome": "Sweatshirt Manga Comprida",
      "categoria": "Roupas",
      "marca": "CozyWear",
      "quantidade_estoque": 130,
      "avaliacao_media": 4.4,
      "numero_avaliacoes": 77
    }
  ]
}
```

Figura 1

3.3. Recomendação por popularidade global

A segunda abordagem implementada foi uma recomendação por popularidade global, usada como base para comparação com a recomendação personalizada. Esta abordagem foi construída sobre a coleção *Purchases*, usando *\$group* para agregar compras por *produto_id* e calcular o total de unidades vendidas, a receita gerada e o número de compras por produto.

Posteriormente, um *\$lookup* sobre *Products* permitiu enriquecer os resultados com o nome e a categoria do produto, seguido de *\$sort* para ordenar pelos mais vendidos e por maior receita. Esta solução é tecnicamente simples e útil para identificar *best sellers*, mas não considera preferências individuais nem histórico do utilizador, sendo por isso menos personalizada.

The screenshot shows the MongoDB Compass interface. On the left, the 'Connections' panel lists the 'CaseStudy | E-commerceDB' connection. The main panel displays the 'Purchases' collection with an aggregation query applied. The query stages are: *\$group*, *\$lookup*, *\$unwind*, *\$sort*, *\$project*, and *\$limit*. The results are displayed as a list of five documents, each representing a product with aggregated sales data.

total_unidades_vendidas	total_receita	numero_compras	produto_id	nome	categoria
6	115.42	3	"p1"	"Camiseta Algodão"	"Roupas"
3	29.700000000000003	2	"p9"	"Meias Pack 5"	"Roupas"
2	175.3	2	"p3"	"Ténis Desportivos"	"Calçado"
2	159	2	"p6"	"Óculos de Sol"	"Acessórios"
2	119.8	2	"p4"	"Mochila Urbânica"	"Acessórios"

Figura 2

3.4. Recomendação por segmento

A terceira recomendação adotou uma lógica de popularidade dentro do mesmo segmento de utilizador, aproximando-se de uma recomendação colaborativa simples. Esta abordagem parte da coleção *Users*, identifica o segmento do utilizador e os produtos já comprados, e depois consulta a coleção *Purchases* para encontrar produtos populares entre utilizadores do mesmo segmento, excluindo os itens já adquiridos pelo utilizador atual.

A pipeline agrupou as compras por *produto_id*, somando quantidade, número de compras e receita. Depois, ligou os resultados ao catálogo *Products* para garantir estoque positivo e devolver informação interpretável, como nome e categoria. Esta recomendação permite explorar afinidades comportamentais entre utilizadores do mesmo segmento e constitui uma alternativa intermédia entre a popularidade global e a recomendação baseada em conteúdo.

The screenshot shows the Compass interface for a MongoDB database named 'CaseStudy | E-commerceDB'. The 'Users' collection is selected, and an aggregation pipeline is being built. The pipeline consists of the following stages: \$match, \$lookup, \$addFields, \$lookup, and \$project. The results are displayed in a JSON format, showing user information and a list of recommended products based on their segment.

```

_id: "u1"
nome: "Ana Sousa"
segmento: "regular"
produtosComprados: Array (3)
  0: "p1"
  1: "p3"
  2: "p9"
recomendacoes_segmento: Array (4)
  0: Object
    total_unidades: 1
    numero_compras: 1
    total_receita: 79.5
    produto_id: "p6"
    nome: "Óculos de Sol"
    categoria: "Acessórios"
  1: Object
    total_unidades: 1
    numero_compras: 1
    total_receita: 59.9
    produto_id: "p4"
    nome: "Mochila Urbânica"
    categoria: "Acessórios"
  2: Object
    total_unidades: 1
    numero_compras: 1
    total_receita: 49
    produto_id: "p10"
    nome: "Fone Bluetooth"
    categoria: "Eletrónica"
  3: Object
    total_unidades: 1
    numero_compras: 1
    total_receita: 39.9
    produto_id: "p8"
    nome: "Sweatshirt Manga Comprida"
    categoria: "Roupas"

```

Figura 3

4. ÍNDICES E OTIMIZAÇÃO DE DESEMPENHO

4.1. Índices criados

Para suportar as consultas e agregações desenvolvidas, foram definidos índices coerentes com os padrões de acesso aos dados. O objetivo foi reduzir custos de leitura e melhorar o desempenho de consultas frequentes, como histórico por utilizador, procura por produto, recomendação por categoria e recomendação por segmento.

Foram considerados adequados os seguintes índices:

```
// Purchases  
  
db.Purchases.createIndex({ user_id: 1, data_compra: -1 })  
db.Purchases.createIndex({ produto_id: 1 })  
db.Purchases.createIndex({ user_segmento: 1, produto_id: 1 })  
  
// Products  
  
db.Products.createIndex({ categoria: 1, avaliacao_media: -1, numero_avaliacoes: -1 })  
db.Products.createIndex({ tags: 1 })  
  
// Users  
  
db.Users.createIndex({ preferencias: 1 })  
db.Users.createIndex({ segmento: 1 })
```

O índice `db.Purchases.createIndex({ user_id: 1, data_compra: -1 })` é adequado para obter rapidamente o histórico de compras de um utilizador, sobretudo quando interessa ordenar por data mais recente. O índice `db.Purchases.createIndex({ produto_id: 1 })` favorece consultas e agregações por produto, como a recomendação por popularidade. O índice `db.Purchases.createIndex({ user_segmento: 1, produto_id: 1 })` apoia diretamente a recomendação por segmento, ao facilitar a filtragem de compras por segmento e produto.

Em *Products*, o índice sobre categoria, *avaliacao_media* e *numero_avaliacoes* é especialmente útil para a recomendação baseada em conteúdo, pois permite filtrar por categoria e ordenar por critérios de qualidade.

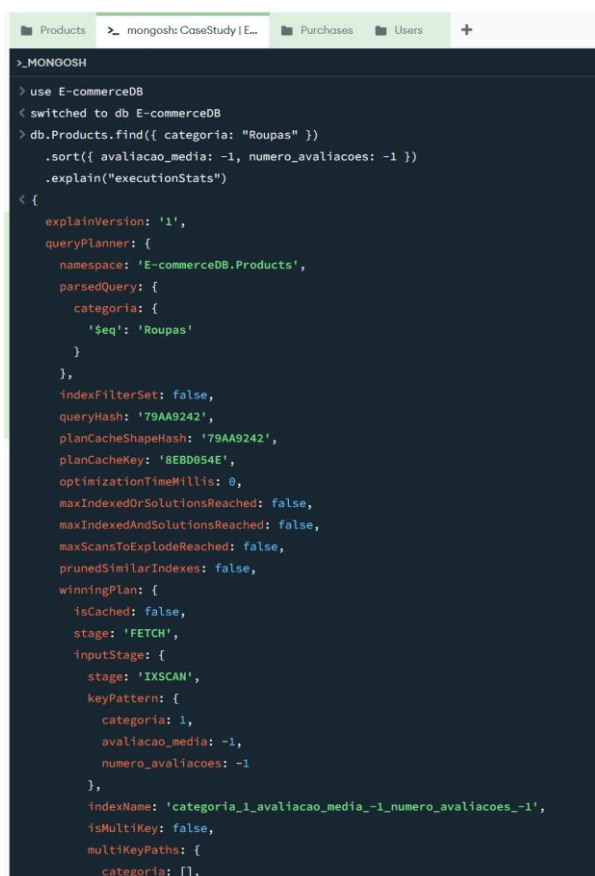
Os índices em *tags*, *preferencias* e *segmento* foram incluídos por coerência com potenciais consultas analíticas ou extensões futuras da lógica de recomendação, como filtragem adicional por afinidade temática ou segmentação comportamental.

4.2. Análise com *explain("executionStats")*

O enunciado recomenda o uso de *.explain("executionStats")* para analisar o comportamento das consultas antes e depois da criação dos índices. A documentação do MongoDB indica que esta abordagem permite observar o plano de execução escolhido, o número de documentos examinados e se a consulta usa índice (IXSCAN) ou faz varrimento integral da coleção (COLLSCAN).

Uma consulta representativa a testar é a seleção de produtos por categoria e ordenação por avaliação:

```
db.Products.find({ categoria: "Roupas" })
.sort({ avaliacao_media: -1, numero_avaliacoes: -1 })
.explain("executionStats")
```

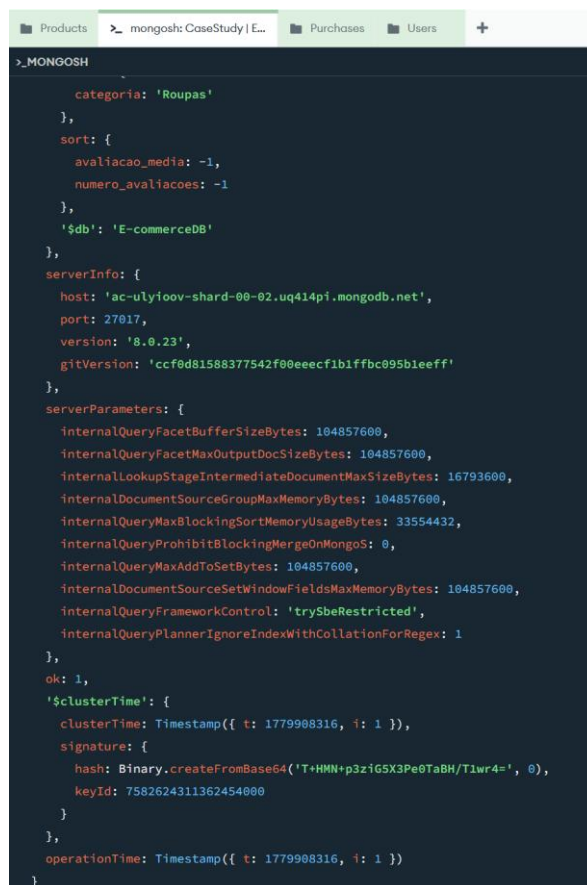


```
> MONGOSH
> use E-commerceDB
< switched to db E-commerceDB
> db.Products.find({ categoria: "Roupas" })
.sort({ avaliacao_media: -1, numero_avaliacoes: -1 })
.explain("executionStats")
{
  explainVersion: '1',
  queryPlanner: {
    namespace: 'E-commerceDB.Products',
    parsedQuery: {
      categoria: {
        '$eq': 'Roupas'
      }
    },
    indexFilterSet: false,
    queryHash: '79AA9242',
    planCacheShapeHash: '79AA9242',
    planCacheKey: '8EBD054E',
    optimizationTimeMillis: 0,
    maxIndexedOrSolutionsReached: false,
    maxIndexedAndSolutionsReached: false,
    maxScansToExplodeReached: false,
    prunedSimilarIndexes: false,
    winningPlan: {
      isCached: false,
      stage: 'FETCH',
      inputStage: {
        stage: 'IXSCAN',
        keyPattern: {
          categoria: 1,
          avaliacao_media: -1,
          numero_avaliacoes: -1
        },
        indexName: 'categoria_1_avaliacao_media_-1_numero_avaliacoes_-1',
        isMultiKey: false,
        multiKeyPaths: {
          categoria: [],

```

(...)

(continuação)



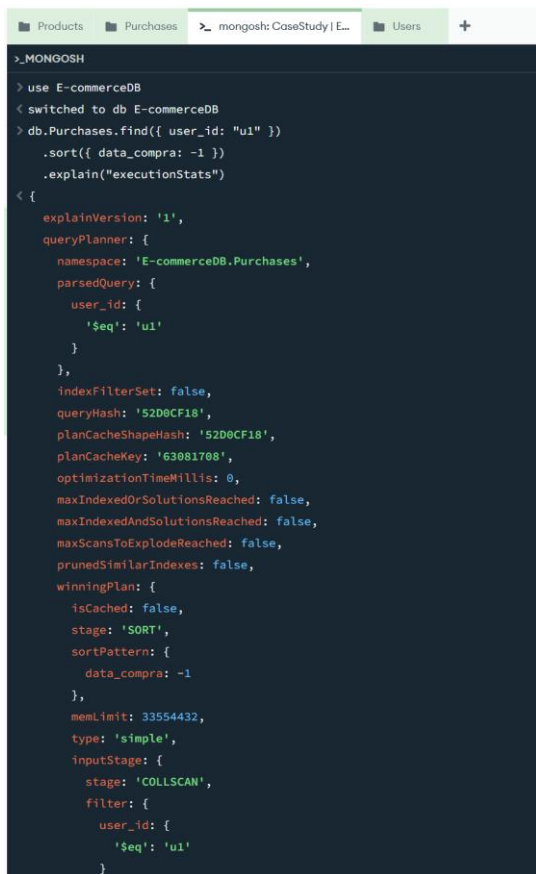
```

    categoria: 'Roupas'
  },
  sort: {
    avaliacao_media: -1,
    numero_avaliacoes: -1
  },
  '$db': 'E-commerceDB'
},
serverInfo: {
  host: 'ac-ulyioov-shard-00-02.uq414pi.mongodb.net',
  port: 27017,
  version: '8.0.23',
  gitVersion: 'ccf0d81588377542f0e0eefc1b1ffbc095b1eeff'
},
serverParameters: {
  internalQueryFacetBufferSizeBytes: 104857600,
  internalQueryFacetMaxOutputDocSizeBytes: 104857600,
  internalLookupStageIntermediateDocumentMaxSizeBytes: 16793600,
  internalDocumentSourceGroupMaxMemoryBytes: 104857600,
  internalQueryMaxBlockingSortMemoryUsageBytes: 33554432,
  internalQueryProhibitBlockingMergeOnMongoS: 0,
  internalQueryMaxAddToSetBytes: 104857600,
  internalDocumentSourceSetWindowFieldsMaxMemoryBytes: 104857600,
  internalQueryFrameworkControl: 'trySbeRestricted',
  internalQueryPlannerIgnoreIndexWithCollationForRegex: 1
},
ok: 1,
'$clusterTime': {
  clusterTime: Timestamp({ t: 1779908316, i: 1 }),
  signature: {
    hash: Binary.createFromBase64('T+HMM+p3z1G5X3Pe0TaBH/T1wr4=', 0),
    keyId: 7582624311362454000
  }
},
operationTime: Timestamp({ t: 1779908316, i: 1 })
}
```

Figura 4

Outra consulta relevante é a obtenção do histórico de compras de um utilizador:

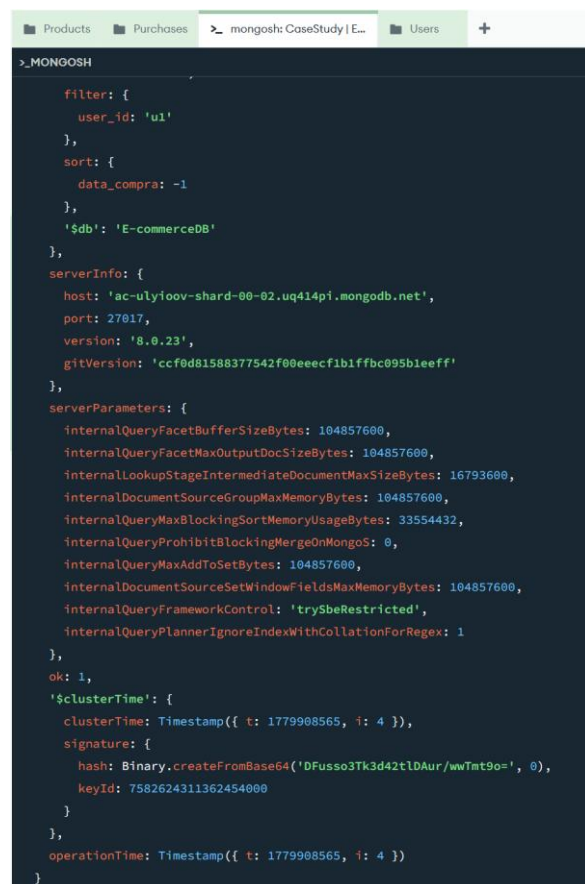
```
db.Purchases.find({ user_id: "u1" })
.sort({ data_compra: -1 })
.explain("executionStats")
```



```
> use E-commerceDB
switched to db E-commerceDB
> db.Purchases.find({ user_id: "u1" })
.sort({ data_compra: -1 })
.explain("executionStats")
{
  explainVersion: '1',
  queryPlanner: {
    namespace: 'E-commerceDB.Purchases',
    parsedQuery: {
      user_id: {
        '$eq': 'u1'
      }
    },
    indexFilterSet: false,
    queryHash: '52D0CF18',
    planCacheShapeHash: '52D0CF18',
    planCacheKey: '63081708',
    optimizationTimeMillis: 0,
    maxIndexedOrSolutionsReached: false,
    maxIndexedAndSolutionsReached: false,
    maxScansToExplodeReached: false,
    prunedSimilarIndexes: false,
    winningPlan: {
      isCached: false,
      stage: 'SORT',
      sortPattern: {
        data_compra: -1
      },
      memLimit: 33554432,
      type: 'simple',
      inputStage: {
        stage: 'COLLSCAN',
        filter: {
          user_id: {
            '$eq': 'u1'
          }
        }
      }
    }
  }
}
```

(...)

(continuação)



```
{
  filter: {
    user_id: 'u1'
  },
  sort: {
    data_compra: -1
  },
  '$db': 'E-commerceDB'
},
serverInfo: {
  host: 'ac-ulyioov-shard-00-02.uq414pi.mongodb.net',
  port: 27017,
  version: '8.0.23',
  gitVersion: 'ccfd81588377542f00eeecf1b1ffbc095b1eeff'
},
serverParameters: {
  internalQueryFacetBufferSizeBytes: 104857600,
  internalQueryFacetMaxOutputDocSizeBytes: 104857600,
  internalLookupStageIntermediateDocumentMaxSizeBytes: 16793600,
  internalDocumentSourceGroupMaxMemoryBytes: 104857600,
  internalQueryMaxBlockingSortMemoryUsageBytes: 33554432,
  internalQueryProhibitBlockingMergeOnMongoS: 0,
  internalQueryMaxAddToSetBytes: 104857600,
  internalDocumentSourceSetWindowFieldsMaxMemoryBytes: 104857600,
  internalQueryFrameworkControl: 'trySbeRestricted',
  internalQueryPlannerIgnoreIndexWithCollationForRegex: 1
},
ok: 1,
'$clusterTime': {
  clusterTime: Timestamp({ t: 1779908565, i: 4 }),
  signature: {
    hash: Binary.createFromBase64('Dfusso3Tk3d42tDAur/wwTmt9o=', 0),
    keyId: 7582624311362454000
  }
},
operationTime: Timestamp({ t: 1779908565, i: 4 })
}
```

Figura 5

Na análise do relatório, deve ser referido que o objetivo dos índices é reduzir *totalDocsExamined* e favorecer o uso de *IXSCAN*, melhorando o custo das consultas e a escalabilidade operacional da solução.

4.3. Testes realizados

O enunciado sugere testar a solução com vários utilizadores, nomeadamente u1, u2, u3, u4 e u5. Esta recomendação foi seguida como estratégia de validação da coerência do sistema de recomendação.

O teste documentado com maior detalhe foi o de *u1*, no qual a recomendação baseada em conteúdo devolveu os produtos *p2* e *p8*. Estes produtos pertencem à categoria *Roupas*, que consta das preferências do utilizador, não estavam presentes no histórico de compras e tinham estoque disponível, o que valida a lógica definida para a recomendação personalizada.

Para os restantes utilizadores, o processo de teste deve confirmar os mesmos critérios: compatibilidade com preferências, exclusão de produtos já comprados, estoque disponível e coerência da ordenação. Esta validação é importante porque demonstra que a pipeline não foi ajustada apenas a um caso isolado, mas responde a um padrão geral de recomendação.

4.4. Escalabilidade e estratégia de *sharding*

Embora o volume de dados do *case study* seja reduzido, o enunciado pede uma reflexão sobre escalabilidade e *sharding*. Numa plataforma real de *e-commerce*, a coleção *Purchases* seria a primeira candidata a *sharding*, pois é a coleção transacional e a que tende a crescer mais rapidamente ao longo do tempo.

Se as consultas continuarem centradas no histórico de compras de cada utilizador, *user_id* é uma *shard key* natural. Contudo, a documentação e as boas práticas do MongoDB indicam que o uso de uma chave *hashed*, como `{ user_id: "hashed" }`, pode ser preferível para distribuir melhor os dados e evitar *hotspots* de escrita concentrados num subconjunto de *shards*.

Em contraste, a coleção *Products* tende a ser menor e mais estável, pelo que não exigiria *sharding* numa fase inicial. Já a escolha de *data_compra* como *shard key* isolada seria desaconselhável, pois datas crescentes podem induzir concentração temporal de escrita e desequilíbrio entre *shards*.

5. DISCUSSÃO CRÍTICA DA SOLUÇÃO

A solução implementada cumpre os objetivos principais do *case study* ao demonstrar uma modelação de dados coerente com MongoDB, uso de padrões de modelação, agregações relevantes e preocupação com desempenho e escalabilidade.

O principal desafio encontrado não esteve na lógica concetual das recomendações, mas sim na necessidade de alinhar rigorosamente os nomes dos campos usados nas agregações com o esquema efetivamente importado nas coleções. Este aspeto teve impacto direto no sucesso dos

estágios *\$lookup*, *\$addFields* e *\$match*, evidenciando que, em MongoDB, a validação empírica do esquema com *findOne()* é um passo indispensável antes de construir *pipelines* mais sofisticadas. A recomendação baseada em conteúdo revelou-se a mais apropriada como solução principal, por ser simples, transparente e coerente com os dados disponíveis. A recomendação por popularidade global serviu como base, enquanto a recomendação por segmento acrescentou uma camada de personalização comportamental baseada em afinidade entre utilizadores do mesmo grupo.

Uma limitação da solução é que o volume reduzido de dados não permite uma avaliação quantitativa robusta do ganho de desempenho com índices nem uma simulação realista de ambiente distribuído. Ainda assim, para fins pedagógicos, o trabalho cobre os aspetos centrais de modelação, consulta, recomendação e escalabilidade pedidos no enunciado.

6. CONCLUSÃO

O desenvolvimento deste *case study* permitiu aplicar, de forma prática, os principais conceitos de modelação documental em MongoDB num cenário simples de e-commerce. A solução final manteve três coleções separadas (*Products*, *Users* e *Purchases*), utilizou referências e subdocumentos de forma apropriada, beneficiou de desnormalização controlada e implementou três estratégias distintas de recomendação: baseada em conteúdo, baseada em popularidade global e baseada em segmento.

A recomendação personalizada revelou-se funcional e coerente com os dados do utilizador, enquanto os índices propostos e a análise com *explain("executionStats")* reforçam a preocupação com desempenho. A reflexão sobre *sharding* mostra ainda que a solução pode ser pensada de forma escalável para cenários com maior volume de utilizadores e transações.

Em síntese, a solução responde aos objetivos do *case study*, demonstrando não apenas a implementação técnica das consultas e agregações, mas também a capacidade de justificar decisões de modelação, otimização e escalabilidade em MongoDB.